Experiment 1: Hardware True Random Number Generator

FPGA Build Challenge 2026 - Beginner level - AMD PYNQ-Z2 (Zynq-7000 XC7Z020)

1. Objective

Computers normally make "random" numbers with a formula (a pseudo-random generator). If someone knows the formula and its starting value, every number can be predicted, which is not acceptable for things like encryption keys. The objective of this experiment is to build a **true** random number generator (TRNG) in the FPGA fabric that gets its randomness from a physical source - the timing jitter of free-running ring oscillators - and to deliver the random bits to the ARM processor over AXI4-Lite. The output is then checked with the NIST frequency and runs tests.

2. Block Diagram

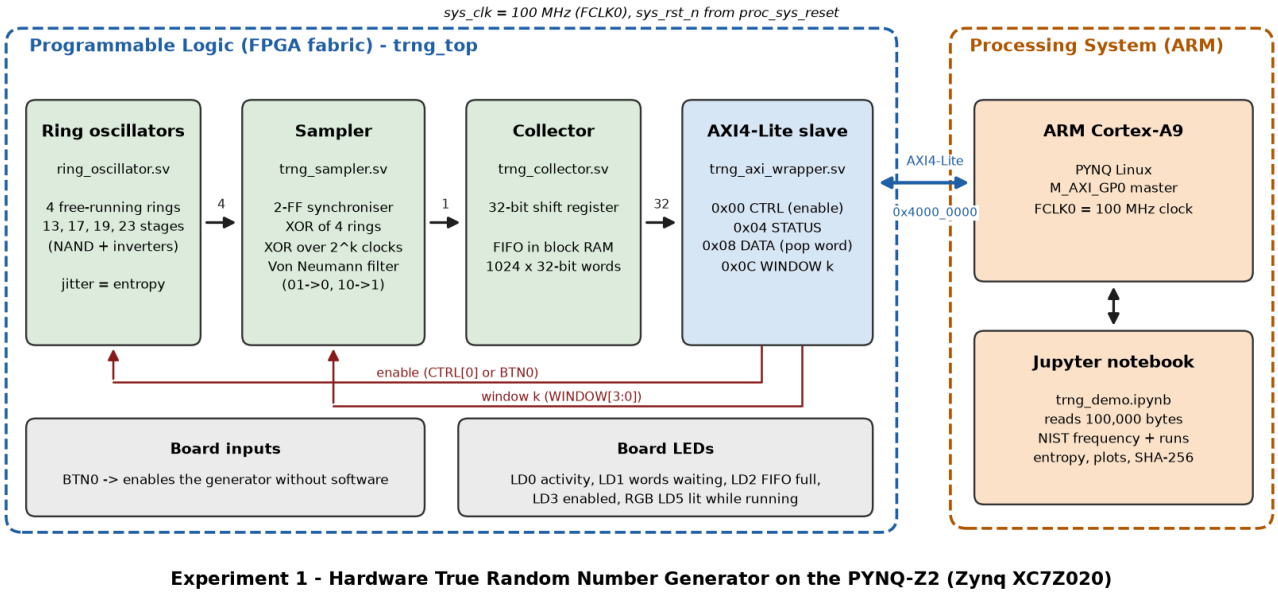


Figure 1 - System block diagram

Four ring oscillators of different lengths run freely inside the FPGA. Their outputs are brought into the 100 MHz clock domain and XORed together. Because samples one clock apart are still correlated, the sampler XORs 2^k samples into one bit (k is set by software, default 6). A Von Neumann filter removes any bias. The collector packs the bits into 32-bit words and stores them in a 1024-word FIFO in block RAM. The ARM processor reads the words through four AXI4-Lite registers, and a Jupyter notebook analyses them.

3. RTL Design

3.1 Module hierarchy

```
system_wrapper                (Vivado block design)
 |-- processing_system7_0     ARM Cortex-A9, gives FCLK0 = 100 MHz and M_AXI_GP0
 |-- rst_ps7_0_100M           reset synchroniser
 |-- ps7_0_axi_periph         AXI interconnect / protocol converter
 +-- trng_top_0               trng_top.sv
      |-- ro_inst             ring_oscillator.sv  (4 x ro_core)
      |-- sampler_inst        trng_sampler.sv
      |-- collector_inst      trng_collector.sv
      +-- axi_inst            trng_axi_wrapper.sv
```

3.2 Modules

Module (file)	What it does
ring_oscillator (ring_oscillator.sv)	Four rings of 13, 17, 19 and 23 stages. Each ring is one NAND gate (used as the enable) followed by inverters in a loop. An odd number of inversions makes the loop oscillate on its own. The keep / dont_touch attributes stop Vivado from removing the loop.
trng_sampler (trng_sampler.sv)	Two flip-flops synchronise the oscillator outputs to the clock. The four bits are XORed into one raw bit per clock. A counter XORs 2^k raw bits into one window bit. A Von Neumann filter takes window bits in pairs: 01 gives 0, 10 gives 1, 00 and 11 are thrown away.
trng_collector (trng_collector.sv)	A 32-bit shift register collects the random bits. Every 32 bits the word is written into a 1024 x 32 FIFO built in block RAM. 11-bit read and write pointers (one extra bit) tell full from empty.
trng_axi_wrapper (trng_axi_wrapper.sv)	AXI4-Lite slave with four registers (table below). Reading the DATA register pops one word from the FIFO. A new read address is only accepted after the previous data has been taken.
trng_top (trng_top.sv)	Connects the four blocks. BTN0 (through a 2-FF synchroniser) can also enable the generator. Drives the LEDs: LD0 blinks with activity, LD1 = words waiting, LD2 = FIFO full, LD3 = enabled, RGB LED LD5 lit while running.

3.3 Register map (base address 0x4000_0000)

Offset	Name	Access	Meaning
0x00	CTRL	R/W	bit 0 = enable the generator
0x04	STATUS	R	bit 0 = FIFO empty, bit 1 = FIFO full, bits 31:16 = words waiting
0x08	DATA	R	next random word; reading removes it from the FIFO
0x0C	WINDOW	R/W	bits 3:0 = k, one raw bit every 2^k clock cycles (reset value 6)

3.4 RTL schematic

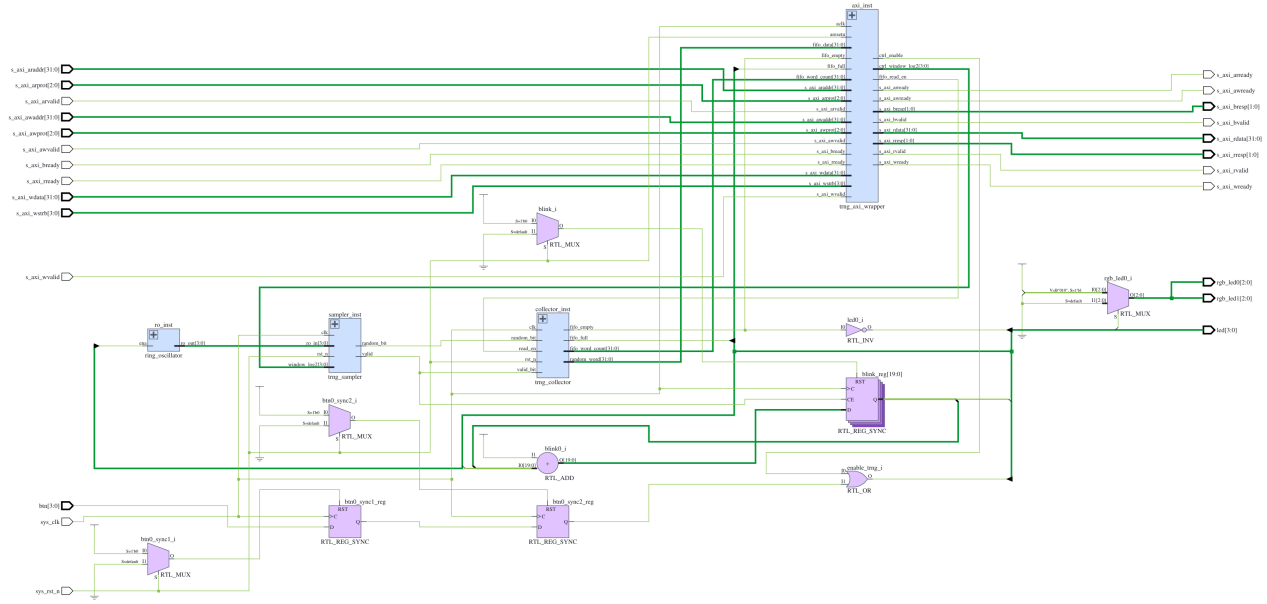


Figure 2 - Elaborated RTL schematic of trng_top (Vivado)

4. Simulation Results

The design was simulated with Icarus Verilog using the testbench *trng_tb.sv*. The testbench checks the register reset value, a register write, an empty-FIFO read, enables the generator, waits 100 us and then reads the status and two random words. All checks passed:

```
status = 004e0000 (78 words waiting)
random words: 3a83365a cb930aca
PASS: TRNG produced random words over AXI
```

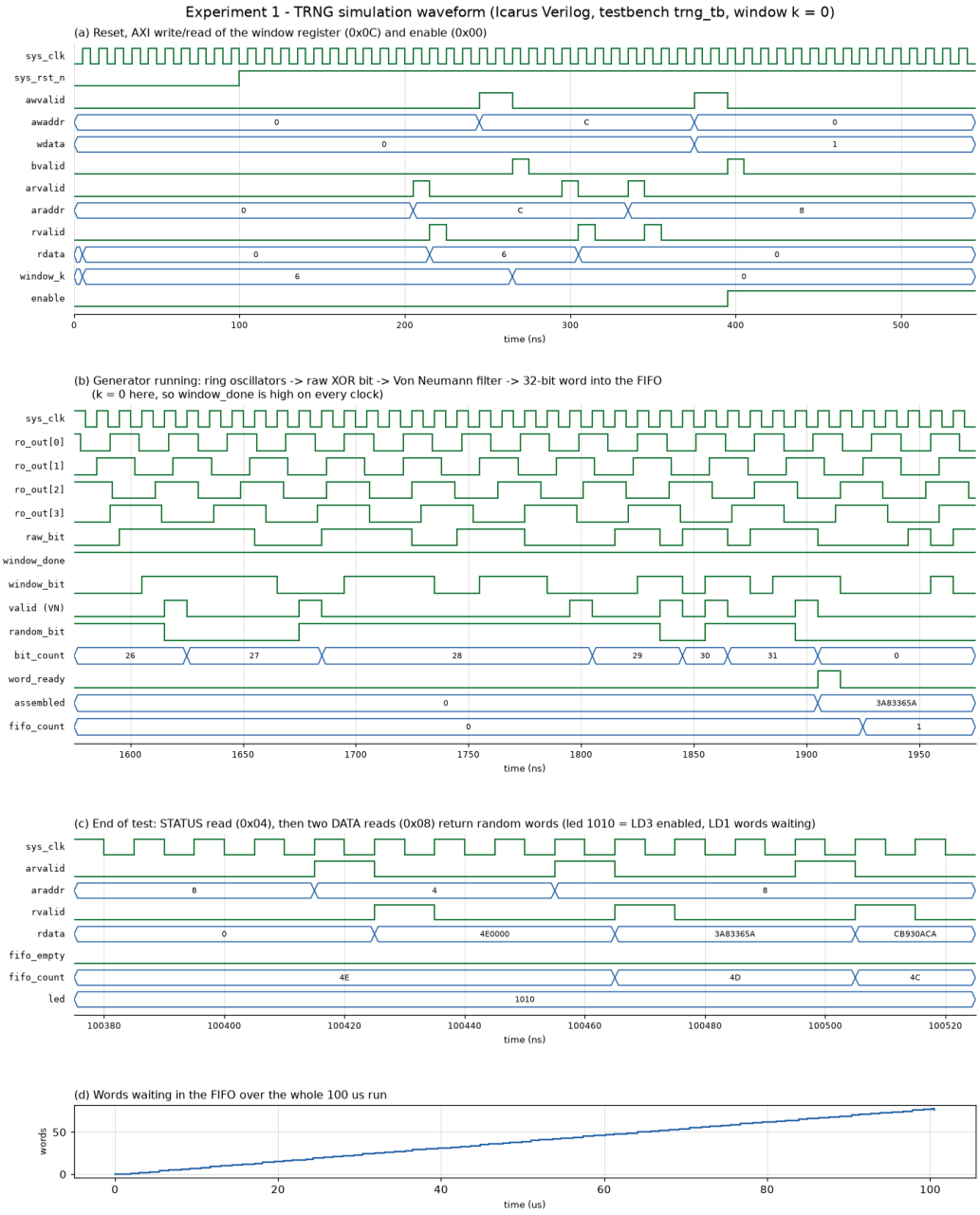


Figure 3 - Simulation waveform (details in Simulation/simulation_report.pdf)

5. Hardware Implementation

5.1 Build

The design was built with Vivado 2026.1 for the xc7z020clg400-1 device using build.tcl. The block design connects the Zynq processing system, a reset block, an AXI interconnect and trng_top. The constraints file (pynq_z2.xdc) assigns the LED and button pins, sets DONT_TOUCH on the ring oscillator cells and ALLOW_COMBINATORIAL_LOOPS on their nets so that the loops pass the design rule check.

Resource	Used	Available	Utilisation
Slice LUTs	571	53,200	1.07 %
Slice registers (flip-flops)	669	106,400	0.63 %
Block RAM tiles (RAMB36)	1	140	0.71 %
Bonded I/O pins	11	125	8.80 %
Global clock buffers	1	32	3.13 %

Timing (100 MHz clock)	Value	Status
Worst setup slack (WNS)	+0.530 ns	**met**
Worst hold slack (WHS)	+0.033 ns	**met**
Failing endpoints	0 of 1656	**met**

5.2 Test on the board

The bitstream was loaded into the PYNQ-Z2, the window was set to k = 6 and the generator was enabled. The 1024-word FIFO filled within 100 ms (LD2 on). 25,000 words (100,000 bytes) were then read from the DATA register and tested with the same code as the Jupyter notebook trng_demo.ipynb.

Test	Result
Bytes tested	100,000
Ones	50.07 % (ideal 50 %)
Shannon entropy	7.9979 / 8 bits per byte
Longest run	20 identical bits
Max |autocorrelation|	0.0062 (lags 1-32)
NIST frequency test	**PASS (p = 0.2386)**
NIST runs test	**PASS (p = 0.2600)**
Repeated 32-bit words	0 of 25,000

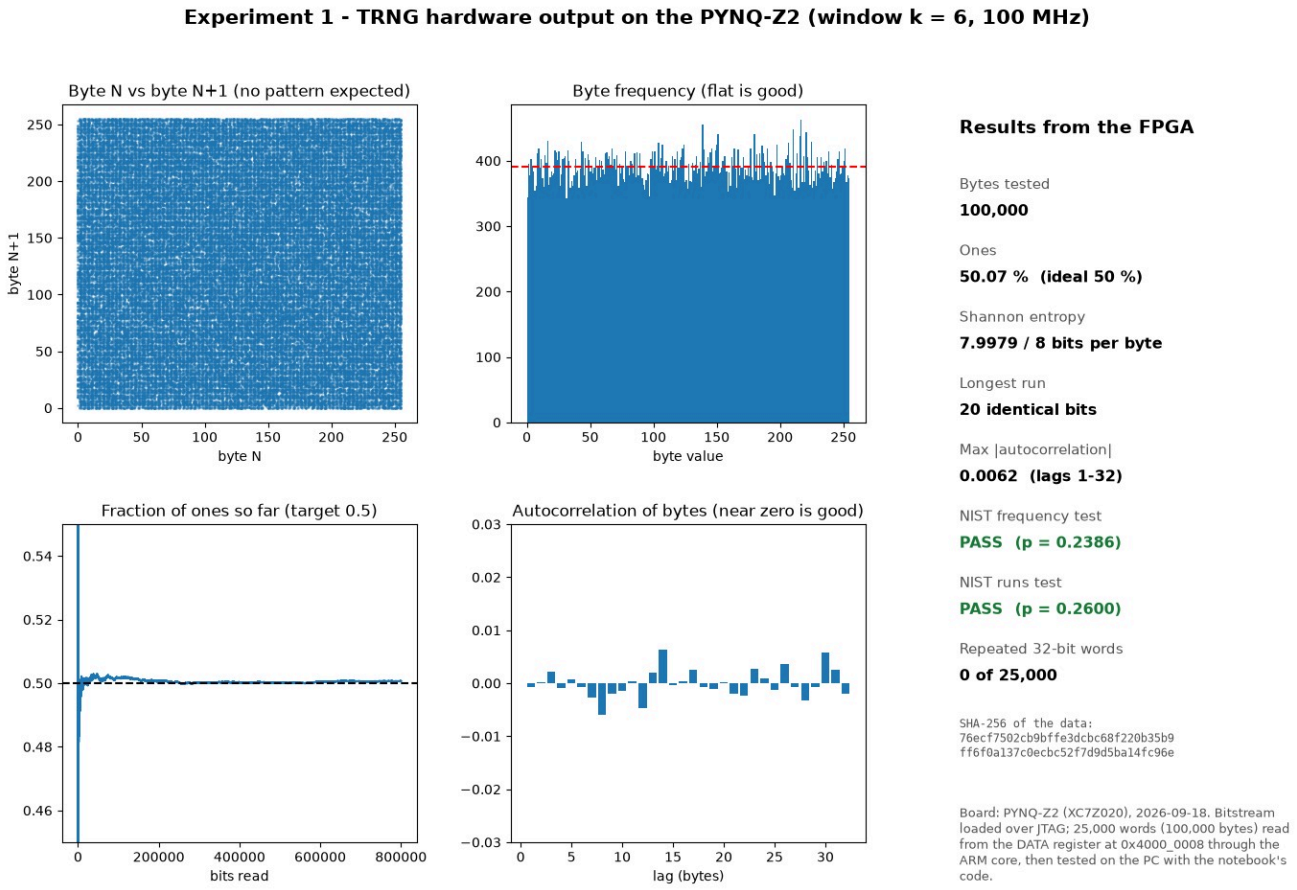


Figure 4 - Random data read from the FPGA and its test results

Run of the Jupyter notebook on the board

The steps of the notebook trng_demo.ipynb (Jupyter folder) were also run on the PYNQ-Z2 with PYNQ, using the same bitstream. tools/execute_all_on_pynq.py runs them on the board over SSH and copies the log and the graph back, and tools/embed_nb_outputs.py puts them into the notebook. It read 100,000 bytes with window k = 6 in 2.0 s and ran the same tests. This is a new set of random bytes, so the numbers differ slightly from the run above, but every test passes again: 50.02 % ones, entropy 7.9982 bits per byte, longest run 17 bits, NIST frequency p = 0.7628 and runs p = 0.5506. The log and the graph the notebook drew are in Jupyter/output.

```
[*] Flashing FPGA bitstream for Experiment 1 (TRNG)...
[+] FPGA Flashed! DONE LED active.
Overlay loaded, TRNG at 0x40000000
Read 100000 bytes in 2.0 s with window k = 6
Ones:           50.02 %
Entropy:        7.9982 / 8.0 bits per byte
Longest run:    17 identical bits
NIST frequency: PASS (p = 0.7628)
NIST runs:      PASS (p = 0.5506)
SHA-256 of the 100,000 bytes: 84aff8366f9fb517edf6d34706c0e748c579ebaee232a52a68a3b44718302f30
```

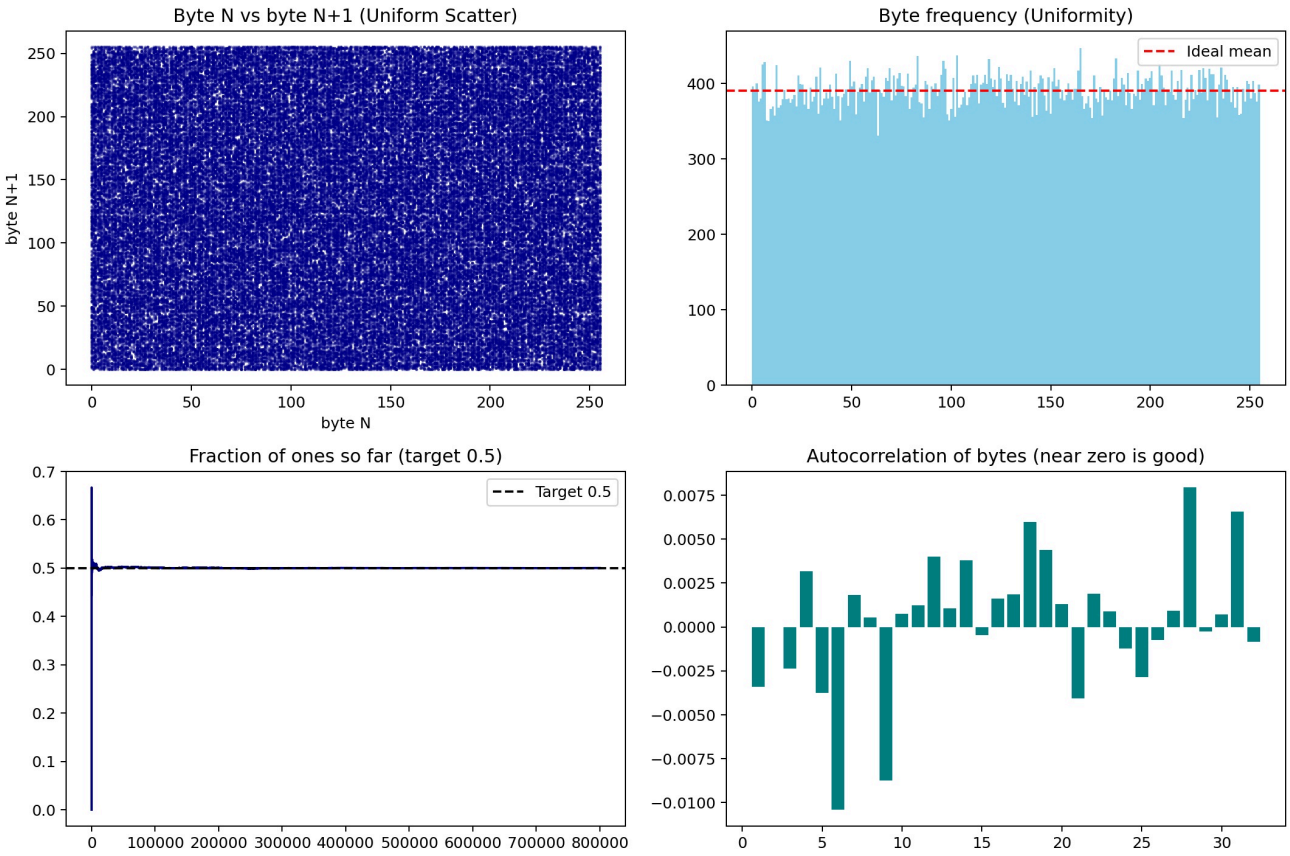


Figure 5 - Graph drawn by trng_demo.ipynb on the board

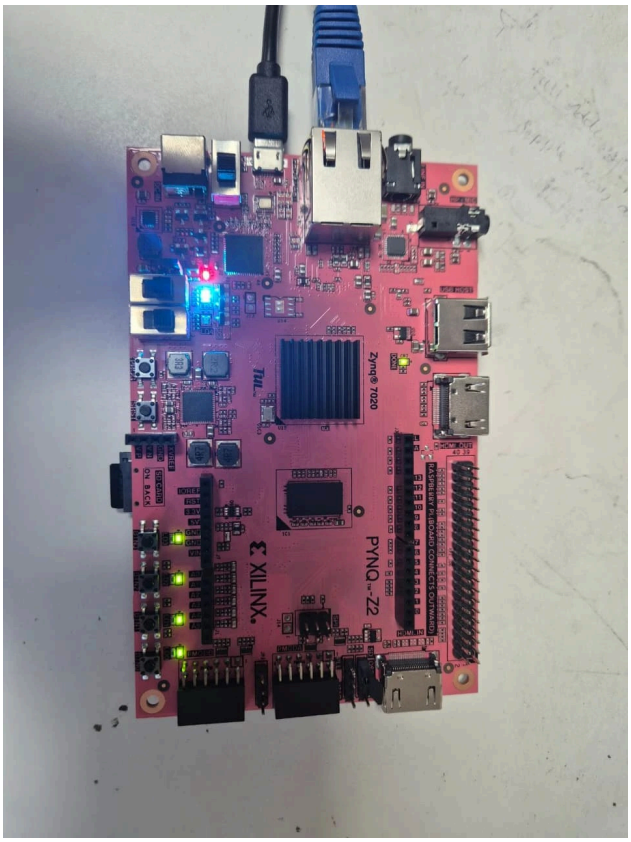


Figure 6 - PYNQ-Z2 board setup: micro-USB (power and JTAG/UART), Ethernet to the PC, PYNQ Linux on the SD card

All plots look as expected for random data: the byte-pair scatter shows no pattern, the byte histogram is flat, the fraction of ones settles at 0.5 and the autocorrelation stays close to zero. Both NIST tests pass (p-value above 0.01).

6. Applications

- **Cryptography:** seeds for encryption keys, nonces and initialisation vectors (after hashing the raw bits, for example with SHA-256).
- **IoT and embedded security:** small devices without a good entropy source can get random numbers from their own FPGA fabric.
- **Authentication:** random challenges for challenge-response login and one-time passwords.
- **Simulation and gaming:** Monte Carlo simulations, lotteries and games that need fair, unpredictable numbers.

7. Conclusion

A working TRNG was designed, simulated and implemented on the PYNQ-Z2. It uses only 1 % of the FPGA's LUTs and one block RAM, meets timing at 100 MHz, and its output passes the NIST frequency and runs tests with 7.998 bits of entropy per byte.

Learning outcomes: how ring oscillators turn electrical noise into random bits and why a synchroniser is needed when sampling them; how a Von Neumann filter removes bias; building a FIFO in block RAM; writing an AXI4-Lite slave and connecting custom logic to the Zynq ARM processor; and testing randomness with statistical tests.

Future scope: add the online health tests from NIST SP 800-90B so that a failed oscillator is detected automatically; add a SHA-256 conditioning block in hardware; stream the data with AXI DMA for higher throughput; and place the oscillators with location constraints so that the entropy is the same after every rebuild.